



PART 2 - PRACTICE

Hands-on Labs

Lab 1: String manipulation (20 min)

Lab 2: Grade classifier (25 min)

Lab 3: Rock-paper-scissors (40 min)

Lab 4: Leap-year calculator (25 min)

Testing and submission (10 min)

Keyboard Input with input()

- `input()` displays a prompt and waits for keyboard input.
- Type a value, then press Enter to continue.
- The result is always a string, even when you type digits.
- Use `int(input(...))` when the lab needs an integer.
- Run the program again to try a different input.

```
name = input("Name: ")
print(f"Hello, {name}!")
# Type Mina, then press Enter.
# Output: Hello, Mina!

score = int(input("Score: "))
print(score + 1)
# Type 85, then press Enter.
# Output: 86

# Enter integer-formatted text for score.
```

f-Strings: Values Inside Text

- `print()` displays a value. The `f` prefix is optional.
- Without `f`, braces and variable names stay literal text.
- With `f` before the quote, `{name}` inserts the current value of `name`.
- Python builds the string first, then `print()` displays it.
- For simple output, `print()` can also separate values with commas.

```
name = "Mina"
score = 85
print("Hello, Mina!")
# Hello, Mina!
print("Hello, {name}!")
# Hello, {name}!
print(f"Hello, {name}!")
# Hello, Mina!
print(f"Score: {score}")
# Score: 85
print("Score:", score)
# Score: 85
```

Lab Workflow: Input to Test

- Identify the input, transformation and required output.
- Write expected results for a small example first.
- Implement one requirement, then run the program.
- Add boundary and invalid cases before submission.
- Use the extra exercises as optional practice within the lab time.

```
# Example plan for strings.py
# Input: "  Python  "
# Clean: "Python"
# Length: 6
# Upper: PYTHON
# Reverse: nohtyP
# First: P

# Empty input is a separate test case.
# Write expected output before coding.
```

Lab 1: String Manipulation

- Create strings.py and read a line of text.
- Remove surrounding whitespace.
- Print its length, uppercase form and reverse.
- Show its first character only when it is nonempty.
- Test a normal word, spaces only and a single character.

```
text = input("Text: ").strip()
print(f"Length: {len(text)}")
print(f"Upper: {text.upper()}")
print(f"Reverse: {text[::-1]}")
if text:
    print(f"First: {text[0]}")
else:
    print("No text entered")

# Input:   Python
# Length: 6, Upper: PYTHON
# Reverse: nohtyP, First: P
```

Lab 1: Step-by-Step Trace

- Trace each intermediate value for a padded word.
- Print transformations of the cleaned text.
- Use an explicit nonempty check before indexing.
- Replace the fixed raw value with input() in the lab.

```
raw = "  Python  "  
text = raw.strip()  
print(f"Length: {len(text)}")  
print(f"Upper: {text.upper()}")  
print(f"Reverse: {text[::-1]}")  
if len(text) > 0:  
    print(f"First: {text[0]}")  
else:  
    print("No text entered")
```

Lab 1: Empty Input & Extension

- Empty input and spaces only both become an empty string.
- The empty case must still print length, upper and reverse.
- Check that indexing never runs for an empty string.
- Optional: report whether cleaned, lowercase text equals its reverse.

```
text = "   ".strip()
print(f"Length: {len(text)}")
print(f"Upper: {text.upper()}")
print(f"Reverse: {text[::-1]}")
if text:
    print(f"First: {text[0]}")
else:
    print("No text entered")

word = "Level".lower()
print(word == word[::-1]) # True
```

Lab 2: Grade Classifier

- Create grades.py. Read an integer score.
- Accept scores from 0 through 100 inclusive.
- A: 90-100, B: 80-89, C: 70-79, D: 60-69, F: 0-59.
- Report an invalid score outside that range.
- Test 59, 60, 89, 90, 100 and 101.

```
score = int(input("Score: "))
if not 0 <= score <= 100:
    print("Invalid score")
elif score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

Lab 2: Boundary Test Design

- A test checks one input against an expected result.
- At each threshold, test the value below and the threshold itself.
- Also test both endpoints and values outside the valid range.
- If 89 prints A, check the comparison operator and threshold.

# Input	Expected
# -1	Invalid score
# 0, 59	F
# 60, 69	D
# 70, 79	C
# 80, 89	B
# 90, 100	A
# 101	Invalid score

Run grades.py once for each input.
Record actual output beside expected.

Lab 2: Optional Pass/Fail Output

- A valid score of 60 or more also receives Pass.
- A valid score below 60 receives Fail.
- Invalid scores must not receive a Pass/Fail result.
- Place the additional decision inside the valid-score branch.

```
score = 59
if not 0 <= score <= 100:
    print("Invalid score")
else:
    # Keep grade classification here.
    if score >= 60:
        print("Pass")
    else:
        print("Fail")

# Test 59, 60 and 101.
```

Lab 3: Rock-Paper-Scissors Rules

- Create rps.py for one round against the computer.
- Accept rock, paper or scissors after `strip().lower()`.
- Equal choices produce a tie.
- Rock beats scissors. Scissors beats paper. Paper beats rock.
- Reject invalid input and print both valid choices.

```
import random

# choice() picks one item.
computer = random.choice(
    ["rock", "paper", "scissors"]
)

# First, test with fixed values:
player = "rock"
computer = "scissors"
# Expected outcome: You win
```

Lab 3: All Nine Valid Pairs

- There are three player choices and three computer choices.
- The nine pairs contain three ties, three wins and three losses.
- Read each row from the player perspective.
- Use these expected results to check every branch.

#	Player	Computer	Result
#	rock	rock	Tie
#	rock	paper	You lose
#	rock	scissors	You win
#	paper	rock	You win
#	paper	paper	Tie
#	paper	scissors	You lose
#	scissors	rock	You lose
#	scissors	paper	You win
#	scissors	scissors	Tie

Lab 3: Branching Example

- Validate the player choice before deciding the outcome.
- Check for a tie before win and loss cases.
- List all three winning combinations.
- Try fixed choices to test every branch.

```
import random
p = input("Your choice: ").strip().lower()
options = ["rock", "paper", "scissors"]
c = random.choice(options)
if p not in ["rock", "paper", "scissors"]:
    print("Invalid choice")
else:
    print(f"You: {p}, computer: {c}")
    if p == c:
        print("Tie")
    elif ((p == "rock" and c == "scissors")
          or (p == "scissors" and c == "paper")
          or (p == "paper" and c == "rock")):
        print("You win")
    else:
        print("You lose")
```

Lab 3: Reproducible Testing

- Temporarily fix the computer choice to make tests repeatable.
- Check padded mixed-case input after normalization.
- Check an invalid word and an empty input separately.
- Restore `random.choice(options)` before submitting.

```
computer = "scissors"
player = "  ROCK ".strip().lower()
print(player, computer)
# rock scissors: expected You win

# With computer fixed to scissors:
# rock      -> You win
# paper     -> You lose
# scissors  -> Tie
# lizard    -> Invalid choice
# empty     -> Invalid choice
```

Lab 4: Leap-Year Rules

- Create leap_year.py for positive integer years.
- Use the Gregorian leap-year rule.
- A year divisible by 400 is a leap year.
- Otherwise, a year divisible by 100 is not.
- Otherwise, a year divisible by 4 is a leap year. All other years are not.

```
# Divisibility uses remainder:  
print(2024 % 4 == 0)    # True  
print(1900 % 100 == 0)  # True  
print(2000 % 400 == 0)  # True
```

```
# Expected classifications:  
# 2024: Leap year  
# 1900: Not a leap year  
# 2000: Leap year  
# 2023: Not a leap year
```

Lab 4: Tracing the Leap-Year Rule

- First identify the results of the divisibility tests.
- A century year needs divisibility by 400.
- An ordinary multiple of 4 is a leap year.
- Explain why 1900 and 2000 have different results.

```
year = 1900
by4 = year % 4 == 0
by100 = year % 100 == 0
by400 = year % 400 == 0
print(by4, by100, by400)
# True True False
print(by400 or (by4 and not by100))
# False

# Repeat with 2000, 2024 and 2023.
# Expected: True, True, False.
```

Lab 4: Leap-Year Calculator

- Reject zero and negative years.
- Combine the rules with and and or.
- Parentheses separate the two ways a year can qualify.
- Print a clear result for the entered year.
- Challenge: express the same rules with if/elif/else.

```
year = int(input("Year: "))
if year <= 0:
    print("Enter a positive year")
else:
    is_leap = (year % 400 == 0 or
               (year % 4 == 0 and
                year % 100 != 0))
    if is_leap:
        print(f"{year}: Leap year")
    else:
        print(f"{year}: Not a leap year")
```

Lab 4: An Equivalent Branch Chain

- Keep the positive-year check first.
- Test divisibility by 400 before divisibility by 100.
- Test divisibility by 100 before divisibility by 4.
- Compare this result with the Boolean-expression version.

```
year = 2000
if year <= 0:
    print("Enter a positive year")
elif year % 400 == 0:
    print(f"{year}: Leap year")
elif year % 100 == 0:
    print(f"{year}: Not a leap year")
elif year % 4 == 0:
    print(f"{year}: Leap year")
else:
    print(f"{year}: Not a leap year")
```

Debugging Review Before Submission

- For each defect, choose an input that exposes it.
- Explain the failure before changing the code.
- After repair, rerun both a normal and a failing case.
- Keep test inputs and expected results in file comments.

```
# Defect                                Revealing test
# Index before guard                    ""
# Missing strip()                       "  "
# Low grade checked first 95
# Missing lower()                       "ROCK"
# Only year % 4 == 0                    1900

# Repairs:
# Guard indexing; normalize text;
# order grade thresholds high to low;
# apply the complete leap-year rule.
```

Testing & Submission

- Run each file and compare its output with the expected result.
- Check boundary values and invalid choices.
- For the game, test all nine valid player/computer pairs.
- Submit strings.py, grades.py, rps.py and leap_year.py.
- Add a comment showing at least three test inputs and their expected results per file.

String input	Expected
" "	No text entered
" A "	Length: 1
Score	Expected
59 / 60	F / D
100 / 101	A / Invalid score
Year	Expected
1900 / 2000	Not leap / Leap
2023 / 2024	Not leap / Leap
0	Positive-year message

Summary & Next Week

- Operators build expressions and comparisons.
- String indices start at 0. Slices exclude the stop index.
- String methods return results. Strings stay immutable.
- if/elif/else chooses the first matching branch.
- Boundary tests help reveal missing cases.

Week 3

Lists and tuples
Dictionaries and sets
for and while loops
range(), break and continue

Preview:
How could you repeat the game
until the player chooses to quit?



Questions?

Python Programming - Week 2